

实 验 报 告

（与程序设计有关的课程用）

课程名称：软件工程

实验题目：实验三　图书管理系统编码设计

班级学号：XXXXXXXXXX

姓　　名：XXX

成　　绩：

大 学

2026 年 10 月 25 日

一、实验目的及要求

（一）实验目的

1. 了解程序设计与编程风格的关系。编程风格不是可有可无的装饰，它直接决定代码是否容易被别人看懂、被自己在一周之后看懂，也决定缺陷是否容易发现和修改。

2. 理解代码撰写规范与程序设计选题质量的关系。选题过大则只能堆砌功能、无暇顾及规范，选题过小则练不到分层与异常处理；只有规模适中的题目，才能把规范真正落到每一行代码上。

3. 掌握一般的代码撰写规范，包括命名、缩进与排版、注释、异常处理、资源管理、输入校验和常量使用等。

4. 熟练掌握一门程序设计语言，独立完成一段完整的编码设计：从问题分析、代码实现到测试验证，全过程独立完成。

（二）实验要求

实验指导书把本次实验的内容分为选作和必做两部分。两部分的要求不同，但都指向同一个目标：通过一段完整的代码，检验设计文档能否被准确地翻译成程序，检验写出的程序是否规范、正确、可维护。

（1）选作部分：写出附录中图书管理系统的信息录入、删除、修改和查询模块的代码。这四个模块恰好覆盖了信息管理类程序最常见的四类操作，也是实验一、实验二中已经实现的功能。本报告从实验二的 library-app 工程中按“界面事件处理—业务逻辑—数据访问—数据库对象”四条线索，把这四个模块的代码完整梳理出来，并说明每一层为什么这样写。

（2）必做部分：输入三个整数 a、b、c，分别作为三角形的三条边，取值范围为 1~100，判断由三条边构成的三角形类型为等边三角形、等腰三角形、一般三角形以及不构成三角形，编写程序代码并画出程序流程图。这是一个典型的“输入—判断—输出”型问题，规模虽小，却包含输入校验、边界值判定和分支顺序等编码设计中最容易出错的地方。

表 1　实验内容与完成情况

序号	实验内容	要求	本报告的完成情况
1	信息录入模块	选作	界面 doInsert → 服务层 checkBook → DAO 调用 sp_book_insert
2	信息删除模块	选作	界面 doDelete（确认框）→ removeBook → 调用 sp_book_delete，数据库拒绝删除在借图书

序号	实验内容	要求	本报告的完成情况
3	信息修改模块	选作	选中行回填表单 → modifyBook → 调用 sp_book_update
4	信息查询模块	选作	关键字模糊查询 → listBooks → PreparedStatement 参数化查询
5	三角形判定程序	必做	Java 21 控制台程序 TriangleClassifier，含 22 条表驱动测试用例
6	程序流程图	必做	图 1，按标准流程图符号绘制，覆盖全部判断分支

（三）关于开发平台的说明

实验指导书给出的软件环境是 WIN 7 操作系统、Access 与 VB 6.0。本次实验沿用实验一、实验二的平台，在 Windows 11 下用 Java 21 完成编码，原因有三点：第一，实验二已经用 Java + JDBC + Swing 实现了图书管理系统，实验三的四个模块代码可以直接从该工程中抽取，前后两个实验的代码风格和设计思路保持一致；第二，Java 的语法约束明确，社区又有成熟的编码规范（命名、异常、资源管理）可以对照，便于讨论“代码撰写规范”这一主题；第三，三角形判定是纯计算问题，用 Java 实现不需要数据库和图形界面，能完整展示输入校验、条件分支和测试代码。实验指导书关注的是编码规范和程序逻辑，并未限定具体语言，因此这样处理不违背实验要求。

二、软、硬件环境

（一）硬件环境

项目	配置
处理器	AMD Ryzen 9 7945HX（16 核 32 线程）
内存	16 GB（系统可用 15.7 GB）
硬盘	Samsung MZVL21T0HCLR-00BL2 固态硬盘，954 GB

（二）软件环境

类别	软件名称	版本
操作系统	Microsoft Windows 11 家庭中文版（64 位）	10.0.26200
开发语言	Oracle JDK（JAVA_HOME = D:\Develop\JDK21）	21.0.9 LTS

类别	软件名称	版本
编译与运行	JDK 自带的 javac / java 命令行工具	21.0.9
数据库服务器	MySQL Community Server	8.0.44
JDBC 驱动	mysql-connector-java（放在 library-app/lib）	8.0.28
数据库客户端	Navicat Premium 17	17.0.4
界面技术	Java Swing（JDK 内置）	JDK 21
测试方式	表驱动测试程序，不依赖 JUnit 等第三方框架	—

图书管理系统工程 library-app 的应用源代码共 19 个 Java 文件，按包分为 util、entity、dao、service、ui 五层；另有 2 个命令行测试与截图工具。三角形程序 experiment3 共 2 个 Java 源文件（TriangleClassifier 与 TriangleTest）。两部分的源码都只使用 JDK 标准库，编译和运行不需要联网。

三、算法或原理分析（实验内容）

3.1 编码设计的任务与步骤

在软件工程中，编码是设计阶段的继续，而不是设计的机械翻译。同一份设计文档，不同的人会写出结构完全不同的代码：有人把所有逻辑塞进一个方法，有人把界面、业务和数据库访问混在一起；前者能运行，后者也能运行，但两三年后需要修改时，前者几乎无法维护。本次实验正是通过两个不同规模的任务，体会“能运行”和“写得好”之间的差距。

本次实验的两项任务在规模上刻意形成对比：

1. 图书管理系统的四个模块属于“在既有工程中补全和整理代码”。工程已有 19 个应用源文件、五个包，四个模块的代码分别落在界面层、业务层和数据访问层，需要沿着调用链把代码读懂、抽出来并解释清楚，重点在于分层是否清晰、异常是否处理得当、SQL 是否集中。

2. 三角形判定程序属于“独立完成一个小程序”。它没有数据库和界面，只有输入、判断和输出，但输入校验、边界值、分支顺序一个都不能少，重点在于逻辑是否严密、测试是否覆盖边界。

两项任务都按下面的步骤完成：第一步，明确输入、输出和约束；第二步，设计处理逻辑或判定顺序并画出流程图；第三步，按命名、排版、注释、异常和资源管理规范写出代码；第四步，设计测试用例并实际运行，记录结果；第五步，回看代码，找出可以改进的地方。

3.2 代码撰写规范

代码撰写规范的内容很多，本次实验不可能逐条罗列，下面只选取对可读性和可维护性影响最大的五个方面：命名、排版与注释、异常处理、资源管理和输入校验，并结合本工程的真实代码说明规范的落实方式。

3.2.1 命名规范

命名是代码中最廉价也最有效的文档。本工程按 Java 社区通行的约定命名：

1. 包名全小写、采用域名倒置形式 com..library，子包按职责划分 util、entity、dao、service、ui。

2. 类名用 UpperCamelCase 且取名词，如 BookDao、LibraryService、TriangleClassifier；工具类加 Util 后缀，如 DBUtil、UIUtil；测试类加 Test 后缀，如 TriangleTest。

3. 方法名用 lowerCamelCase 且以动词开头，如 search、insert、update、delete、checkBook、findOpenLoans；返回布尔值的方法用 is/has 开头，如 isBlank。

4. 变量名用有意义的名词，如 bookNo、cardNo、openLoans；集合用 list、map 作后缀，如 books、publisherMap；循环变量 i、j 只在极小的作用域内使用。

5. 常量全部大写并用下划线分隔，如 MIN_SIDE、MAX_SIDE、COLS、DETAIL_COLS、FMT；用常量代替散落在代码中的魔法值。

6. 数据库对象统一使用小写下划线命名，并用前缀区分类型：表 book、reader、borrow；存储过程 sp_book_insert、sp_return_book；视图 v_borrow_detail、v_book_stat；触发器 trg_borrow_before_insert；外键 fk_borrow_book。

有两点需要说明。第一，三角形程序中三条边命名为 a、b、c，看起来不符合“有意义命名”的原则，但它们与数学题目中的记号一致，读者一看就知道指三条边，属于约定俗成的例外；若改成 side1、side2、side3，反而显得啰嗦。第二，命名要避免无意义的缩写和拼音，例如把 borrowDate 写成 jyrq、把 reader 写成 dz，短期省事，长期增加阅读成本。规范的价值在于让读者把注意力放在逻辑上，而不是先猜名字的含义。

3.2.2 排版与注释

排版方面，本工程统一采用 4 个空格缩进（不使用 Tab），左大括号不另起一行（K&R 风格），运算符两侧留空格，逗号后留空格，一行只写一条语句；类的字段、构造器、公有方法、私有方法之间用空行分开。这样排版的代码在阅读时自然形成“块”，眼睛不需要逐行扫描就能找到方法边界。

注释方面分为三个层次：第一层是类和接口的 Javadoc，说明这个类负责什么、和其他类是什么关系；第二层是公有方法的 Javadoc，说明参数、返回值和可能抛出的异常；第三层是方

法内部的行内注释，只在逻辑不直观时解释“为什么这样做”，不复述代码“做了什么”。以数据库连接工具类 DBUtil 为例，类注释说明它“从 classpath 下的 db.properties 读取连接参数，加载驱动并对外提供 getConnection()”，读者不必看实现就知道它的职责。

```
/**
 * JDBC 连接管理工具类。
 *
 * <p>从 classpath 下的 db.properties 读取连接参数，加载驱动并对外提供
 * {@link #getConnection()}。所有 DAO 都通过它取得数据库连接。</p>
 */
public final class DBUtil {
```

行内注释则用来解释容易被忽略的细节。例如 TriangleClassifier 在判断三角形不等式时特意加了注释，说明为什么要把 int 转成 long，以及为什么用“和小于等于第三边”而不是“和小于第三边”；这类注释记录的是设计理由，删掉代码可以运行，但后来的人很可能会把 long 改回 int，或用错边界条件。

```
// 第二步：三角形不等式。用“和 <= 第三边”判否，可覆盖退化三角形（如 1,2,3）。
// 采用 long 运算防止极端输入下 int 加法溢出。
if ((long) a + b <= c || (long) a + c <= b || (long) b + c <= a) {
    return NOT_TRIANGLE;
}
```

注释还要求与代码同步修改。工程中保留的注释都是当前实现仍然成立的说明；过时的注释比没有注释更危险，因为它会误导读者。

3.2.3 异常处理与资源管理

Java 的受检异常机制要求调用者显式处理 SQLException，这为数据库程序提供了统一的错误处理入口。本工程的做法是：数据访问层不吞异常，一律把 SQLException 向上抛出；业务逻辑层把“业务规则不满足”的情况包装成自定义的 BusinessException，把数据库异常原样上抛；界面层用一个 catch (Exception) 统一捕获，弹出中文提示。这样界面代码里看不到零散的 try-catch，却能保证每一种失败都有提示。

资源管理使用 Java 7 引入的 try-with-resources 语法。Connection、PreparedStatement、CallableStatement、ResultSet 都实现了 AutoCloseable，写在 try 的括号里，无论正常返回还是抛出异常，JVM 都会按声明的逆序自动关闭，不需要手写 finally，也不会因为异常而泄漏连接。

```
public List<Book> search(String keyword) throws SQLException {
        boolean has = keyword != null && !keyword.isBlank();
        String sql = "SELECT " + COLS + " FROM book"
                   + (has ? " WHERE book_no LIKE ? OR title LIKE ? OR author LIKE ? OR publisher
LIKE ?" : "")
                + " ORDER BY book_no";
```

```
        List<Book> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            if (has) {
                String like = "%" + keyword.trim() + "%";
                for (int i = 1; i <= 4; i++) {
                    ps.setString(i, like);
                }
            }
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(map(rs));
                }
            }
        }
        return list;
    }
```

对于确实需要忽略异常的场合（例如关闭资源时的二次异常），必须显式写出忽略的理由。DBUtil 的 close 方法在每个 catch (SQLException ignored) 中保留空块，是因为关闭失败无法补救，不能让它覆盖掉真正的业务异常；这种“有意的忽略”与“忘记处理”在代码评审时必须能区分开。

3.2.4 防御式编程与输入校验

程序的错误大多来自边界输入。本工程在三处设置校验：界面层用 trim 去掉首尾空格，价格用 BigDecimal 解析，解析失败抛出带示例的 IllegalArgumentException；服务层检查必填项、价格非负、性别只能取“男”或“女”等业务规则；数据库层再用主键、外键、ENUM 取值和触发器兜底。三层校验并不重复：界面层拦住明显的空值和格式错误，服务层拦住业务规则，数据库层保证任何入口写入的数据都合法。

三角形程序同样遵循防御式编程：先检查输入个数是否为 3，再检查每个 token 能否解析为整数，然后检查三条边是否都落在 1~100 的闭区间内，最后才进行三角形判定。顺序不能颠倒——如果先做三角形不等式，输入 0、-1 这类非法值可能因为不满足不等式而被误判为“不构成三角形”，掩盖了输入本身不合法的事实。

3.2.5 规范的落实情况

表 2　代码撰写规范的落实情况

规范项	具体要求	本工程实例
命名	包全小写；类 UpperCamelCase；方法 lowerCamelCase；常量全大写	com.library.library；BookDao；removeBook；MIN_SIDE

排版	4 空格缩进；K&R 大括号；一行一条语句；字段与方法间空行	全部源文件统一格式，缩进无 Tab
注释	类与公有方法写 Javadoc；行内注释解释设计理由；中文注释	DBUtil 类注释；TriangleClassifier 的 long 转换注释
异常	不吞异常；业务异常与数据库异常分开；界面统一中文提示	BusinessException；UIUtil.error
资源	try-with-resources 自动关闭；确有忽略时写明理由	BookDao.search；DBUtil.close
校验	界面 trim、服务层规则校验、数据库约束三层防护	checkBook；checkReader；触发器
常量	用有名字的常量代替魔法值	MIN_SIDE/MAX_SIDE；COLS；DETAIL_COLS
输出	界面提示准确；命令行程序用退出码区分错误类型	退出码 0/1/2/3

3.2.6 编程风格与选题质量的关系

本次实验对“编程风格与选题质量的关系”有了切身体会。图书管理系统有 19 个应用源文件、五层结构，如果不统一命名和分层，界面代码里混入 SQL、DAO 里弹出对话框，功能再多也无法维护；反过来，规范一旦定下来，新增一个读者管理模块只要照搬图书模块的结构，代码量不大却能保证一致。三角形程序只有两百行，看起来与“选题质量”关系不大，但正是这个小题目把输入校验、边界值、分支顺序三个最容易出错的点集中暴露出来；如果直接拿一个几千行的系统练手，反而看不清这些细节。可见选题大小要服务于训练目标：大题目练结构和分层，小题目练逻辑和测试。

3.3 图书管理系统四个模块的代码设计

实验二的程序采用表示层、业务逻辑层、数据访问层、数据库层四层结构，层与层之间只能由上向下调用。四个信息管理模块虽然功能不同，调用链却是同构的：界面上的按钮触发事件处理方法，事件方法收集表单数据后调用业务方法，业务方法做规则检查后调用数据访问方法，数据访问方法通过 JDBC 执行 SQL 或存储过程。表 3 列出了四个模块的调用链。

表 3 四个模块的调用链

模块	界面事件	业务方法	数据访问方法	数据库对象
录入	BookPanel.doInsert	addBook → checkBook	BookDao.insert	sp_book_insert
删除	BookPanel.doDelete	removeBook	BookDao.delete	sp_book_delete

模块	界面事件	业务方法	数据访问方法	数据库对象
修改	BookPanel.doUpdate	modifyBook → checkBook	BookDao.update	sp_book_update
查询	BookPanel.reload	listBooks	BookDao.search	book 表（PreparedStatement）

四个模块遵循相同的约定：第一，界面代码中不出现任何 SQL 语句，只负责收集输入、显示结果和提示错误；第二，查询使用 PreparedStatement 并把用户输入全部作为参数绑定，录入、修改、删除使用 CallableStatement 调用存储过程；第三，结果集在 DAO 内部转换为实体对象，界面只和实体对象打交道；第四，所有异常都在界面事件方法中统一处理，不在 DAO 中弹对话框。读者信息管理模块与图书模块结构完全一致，只是字段和存储过程名不同，这种一致性本身就是可维护性的一部分。

3.4　三角形判定问题的分析

题目要求输入三个整数 a、b、c 作为三角形的三条边，取值范围为 1~100，判断由三条边构成的三角形是等边三角形、等腰三角形、一般三角形，还是不构成三角形。输入是三个整数，输出是四种判定结果之一；约束有两条：三条边都必须是整数，且都落在 1~100 的闭区间内。

判定逻辑分为四步，顺序不能交换：

1. 第一步，输入校验。检查是否恰好输入了 3 个整数，每个 token 能否解析为整数；再检查 a、b、c 是否都在 1~100 范围内。

2. 第二步，三角形不等式。任意两边之和必须严格大于第三边，即 a+b>c、a+c>b、b+c>a 同时成立；只要有一条不成立，就是“不构成三角形”。程序用“和小于等于第三边”判否，这样 1+2=3、1+1=2 这类退化三角形也能被正确排除。

3. 第三步，判断等边。三边相等（a=b 且 b=c）时输出“等边三角形”。

4. 第四步，判断等腰与一般。恰有两边相等（a=b 或 b=c 或 a=c）时输出“等腰三角形”，否则输出“一般三角形”。

这里有三个容易出错的地方，需要特别说明。第一，范围校验必须放在三角形不等式之前。若先判不等式，输入 0、3、4 时 0+3<4 会先被判定为“不构成三角形”，看起来也“没有崩溃”，但实际掩盖了输入越界这一更根本的错误；题目明确限定取值范围，就应当先报告“输入越界”。第二，退化三角形必须用“小于等于”而不是“小于”排除，1、2、3 满足 1+2=3，不能构成三角形。第三，等边必须在等腰之前判断，否则三边相等的等边三角形会先命中“两边相等”的分支而被误报为等腰三角形。

为了系统地设计测试用例，先对输入进行等价类划分。有效等价类按输出结果划分，无效等价类按违反的约束划分，结果如表 4 所示。

表 4　三角形判定问题的等价类划分

编号	输入条件	类型	期望结果
E1	三条边全等且能构成三角形（如 5,5,5）	有效	等边三角形
E2	恰有两边相等且能构成三角形（如 5,5,8）	有效	等腰三角形
E3	三边互不相等且能构成三角形（如 3,4,5）	有效	一般三角形
E4	存在小于 1 的边（如 0,3,4；-3,4,5）	无效	输入越界
E5	存在大于 100 的边（如 101,50,50）	无效	输入越界
E6	存在非整数输入（如 abc、3.5）	无效	提示不是整数
E7	输入个数不是 3 个（如 3,4 或 3,4,5,6）	无效	提示个数错误
E8	任两边之和小于或等于第三边（如 1,2,3）	无效	不构成三角形

在等价类的基础上再取边界值。边长的有效范围是 1~100，因此对每个变量取 0、1、2、99、100、101 等边界及其组合；判定条件的边界取“两边之和等于第三边”和“两边之和比第三边大 1”两类。表 5 列出了主要的边界值用例，表 6 中的测试用例完整覆盖了这些等价类和边界值。

表 5　主要边界值用例

边界类型	取值	说明
范围下界	1,1,1；1,2,2；1,100,100	三边均取下界、最短边取下界
范围上界	100,100,100；100,99,98；100,100,1	三边均取上界及接近上界
下界外	0,3,4；0,0,0	比下界小 1，应报越界
上界外	101,50,50	比上界大 1，应报越界
退化三角形	1,2,3；1,1,2；100,50,50	两边之和等于第三边，应判不构成
合法临界	1,2,2；1,100,100	最短边取 1 时恰好满足不等式

三角形判定程序的时间复杂度为 O(1)：无论输入取何值，最多执行常数次比较；空间复杂度也是 O(1)，只用了三个整型变量和几个中间量。程序流程图如图 1 所示。

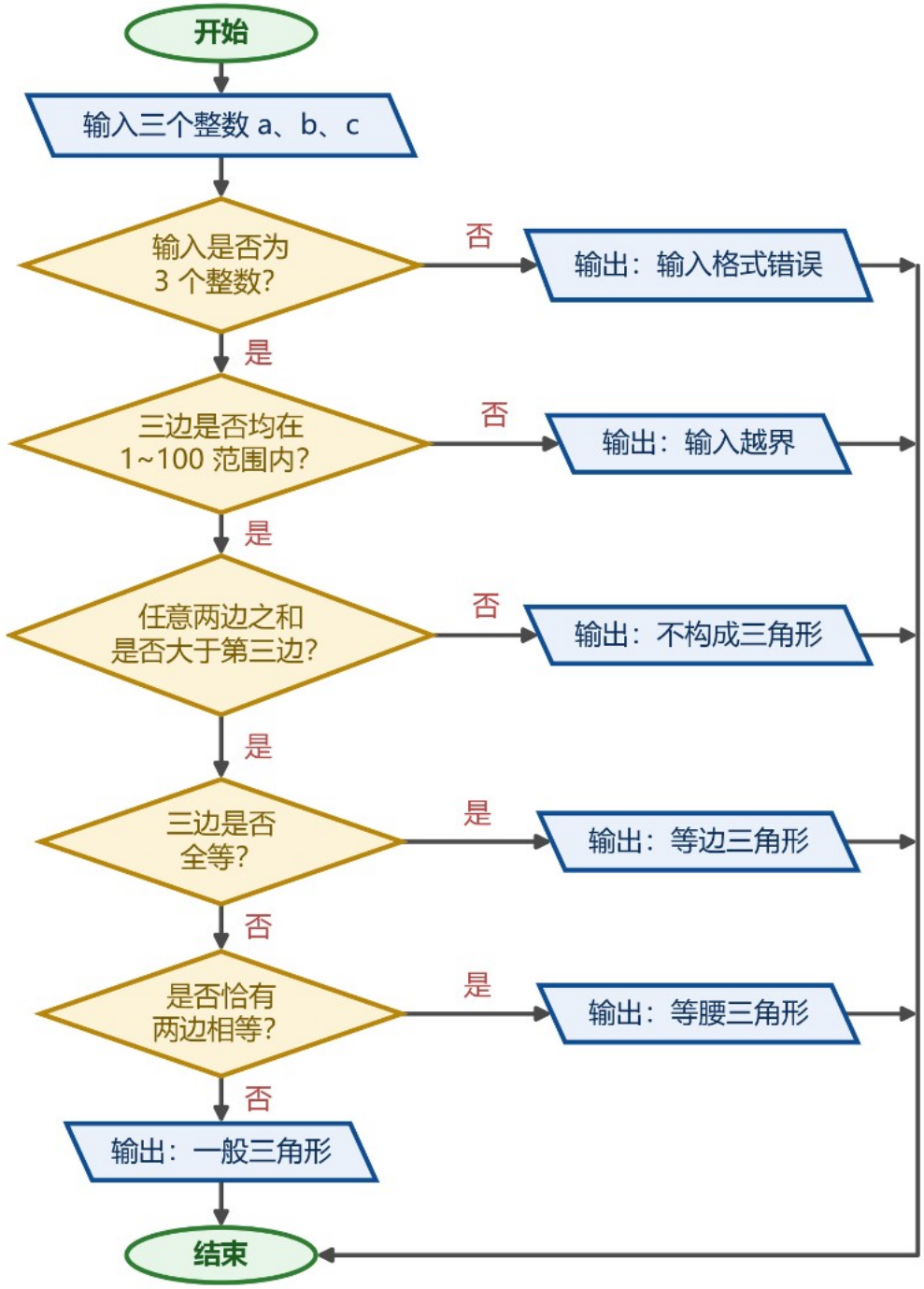


图 1　三角形判定程序流程图

流程图使用标准符号：椭圆表示开始和结束，平行四边形表示输入和输出，菱形表示判断。程序先判断“输入是否为 3 个整数”和“三边是否均在 1~100 范围内”，任一条件不满足就输出相应的错误信息并结束；再判断“任意两边之和是否大于第三边”，为“否”时输出“不构成三角形”；三条判断都通过后，依次判断“三边是否全等”“是否恰有两边相等”，是则分别输出等边三角形、等腰三角形，否则输出一般三角形。图中的每条路径都能与 classify 方法的返回语句一一对应。

四、程序代码或实现过程

4.1　查询模块

查询是使用频率最高的操作。图书管理界面的查询工具条只提供一个关键字输入框，程序把关键字同时匹配书号、书名、作者和出版社四个字段，采用的是“包含”匹配而不是精确匹配，因此管理员不必记住完整的书号。界面事件方法 reload 调用业务方法 listBooks，后者直接转给 BookDao.search。数据访问层的实现如下。

```
public List<Book> search(String keyword) throws SQLException {
        boolean has = keyword != null && !keyword.isBlank();
        String sql = "SELECT " + COLS + " FROM book"
                   + (has ? " WHERE book_no LIKE ? OR title LIKE ? OR author LIKE ? OR publisher
LIKE ?" : "")
                + " ORDER BY book_no";
        List<Book> list = new ArrayList<>();
        try (Connection conn = DBUtil.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            if (has) {
                String like = "%" + keyword.trim() + "%";
                for (int i = 1; i <= 4; i++) {
                    ps.setString(i, like);
                }
            }
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(map(rs));
                }
            }
        }
        return list;
    }
```

这段代码体现了查询模块的三个要点。第一，SQL 语句的“骨架”是固定的，只有“是否带 WHERE”这一处根据关键字是否为空动态拼接，用户输入的值一律用 ？占位符绑定，既避免了 SQL 注入，也省去了手工转义引号的麻烦。第二，LIKE 的通配符只在参数值两侧添加一次，四个字段共用同一个参数值，代码不重复。第三，Connection、PreparedStatement、ResultSet 都写在 try-with-resources 中，发生异常时按顺序自动关闭，不会泄漏连接。界面侧的 reload 方法只负责刷新表格和状态栏，不涉及任何 SQL。

```
public void reload() {
        try {
```

```
            List<Book> books = service.listBooks(kwField.getText());
            SwingUtilities.invokeLater(() -> {
                model.setRowCount(0);
                for (Book b : books) {
                    model.addRow(new Object[]{b.getBookNo(), b.getTitle(), b.getAuthor(),
                            b.getPrice(), b.getPublisher(), b.getSummary()});
                }
                owner.refreshStatus();
            });
        } catch (Exception ex) {
            UIUtil.error(this, "查询图书失败：\n" + ex.getMessage());
        }
    }
```

4.2 录入模块

录入模块的处理流程是：用户填写表单，单击“录 入”按钮，事件方法 doInsert 先调用 readForm 把文本框内容组装成 Book 对象，再交给业务方法 addBook；addBook 先调用 checkBook 做必填项和价格合法性检查，通过后调用 BookDao.insert 执行存储过程 sp_book_insert。代码分成三层，每层只做一件事。

```
 private void doInsert() {
        try {
            service.addBook(readForm());
            UIUtil.info(this, "图书录入成功。");
            clearForm();
            reload();
        } catch (Exception ex) {
            UIUtil.error(this, "录入失败：\n" + ex.getMessage());
        }
    }
 private void checkBook(Book b) throws BusinessException {
        if (isBlank(b.getBookNo())) {
            throw new BusinessException("书号不能为空。");
        }
        if (isBlank(b.getTitle())) {
            throw new BusinessException("书名不能为空。");
        }
        if (isBlank(b.getAuthor())) {
            throw new BusinessException("作者不能为空。");
        }
        if (b.getPrice() == null || b.getPrice().compareTo(BigDecimal.ZERO) < 0) {
            throw new BusinessException("价格必须是不小于 0 的数字。");
```

```
        }
    }
public void insert(Book b) throws SQLException {
        try (Connection conn = DBUtil.getConnection();
             CallableStatement cs = conn.prepareCall("{call sp_book_insert(?,?,?,?,?,?)}")) {
            cs.setString(1, b.getBookNo());
            cs.setString(2, b.getTitle());
            cs.setString(3, b.getAuthor());
            cs.setBigDecimal(4, b.getPrice());
            cs.setString(5, b.getPublisher());
            cs.setString(6, b.getSummary());
            cs.execute();
        }
    }
```

录入模块的关键设计有两点。第一，界面层只做“收集—调用—反馈”三件事，必填项和价格的检查放在业务层，这样无论从界面录入还是从命令行测试程序录入，都会走同一套校验。第二，主键是否重复、价格是否超出 DECIMAL(8,2) 的表示范围等检查交给数据库完成：sp_book_insert 直接执行 INSERT，主键冲突时 MySQL 返回 1062 错误，JDBC 把它包装成 SQLException，界面统一捕获并把数据库返回的中文或英文消息显示给用户。程序没有试图在 Java 里重新实现数据库已经做好的检查，避免了规则不一致。读者录入模块结构完全相同，只是界面上的性别、类型、级别使用下拉框，从输入方式上就限制了取值范围。

4.3 修改模块

修改模块与录入模块共用大部分代码，区别在于数据来源和调用的存储过程。用户在表格中单击某一行时，表格的列表选择监听器会调用 fillForm 把该行数据回填到下方表单，管理员只需修改需要变更的字段，不必重新输入书号。单击“修 改”后，doUpdate 调用 modifyBook，后者复用 checkBook 做同样的合法性检查，再调用 BookDao.update 执行 sp_book_update。

```
private void fillForm(int row) {
        noField.setText(str(row, 0));
        titleField.setText(str(row, 1));
        authorField.setText(str(row, 2));
        priceField.setText(str(row, 3));
        pubField.setText(str(row, 4));
        summaryArea.setText(str(row, 5));
    }
private void doUpdate() {
        try {
            service.modifyBook(readForm());
```

```
            UIUtil.info(this, "图书信息修改成功。");
            reload();
        } catch (Exception ex) {
            UIUtil.error(this, "修改失败：\n" + ex.getMessage());
        }
    }
public void update(Book b) throws SQLException {
        try (Connection conn = DBUtil.getConnection();
             CallableStatement cs = conn.prepareCall("{call sp_book_update(?,?,?,?,?,?)}")) {
            cs.setString(1, b.getBookNo());
            cs.setString(2, b.getTitle());
            cs.setString(3, b.getAuthor());
            cs.setBigDecimal(4, b.getPrice());
            cs.setString(5, b.getPublisher());
            cs.setString(6, b.getSummary());
            cs.execute();
        }
    }
```

修改模块复用了录入模块的校验逻辑，这是分层设计带来的直接好处：如果校验代码写在按钮事件里，录入和修改就要各写一遍，改规则时容易漏改一处。需要说明的是，本程序以书号作为定位键，存储过程 sp_book_update 用 WHERE book_no = p_no 定位记录并更新其余字段；书号是主键，正常情况下不应修改，界面虽然保留了书号输入框，但实际使用中只读不写。如果确实需要更换书号，应当先删除旧记录再重新录入，否则 UPDATE 语句匹配不到任何行，虽然不会报错，数据也不会改变。这一点是程序可以改进的地方，后文 5.4 节还会讨论。

4.4 删除模块

删除是不可逆操作，因此界面在真正删除之前先做两次检查：一是书号不能为空，二是弹出确认对话框让用户确认。确认后调用 removeBook，最终由 BookDao.delete 执行 sp_book_delete。删除操作在数据库端有两层保护：存储过程先检查该书是否存在未归还的借阅记录，若有则用 SIGNAL 抛出“该图书仍在借阅中，不能删除”；即使没有未归还记录，只要该书有过历史借阅，外键 fk_borrow_book（ON DELETE RESTRICT）也会阻止删除，从而保证借阅历史不被破坏。

```
private void doDelete() {
        String no = UIUtil.trim(noField.getText());
        if (no.isEmpty()) {
            UIUtil.error(this, "请先在表格中选择要删除的图书。");
            return;
        }
        if (!UIUtil.confirm(this, "确定要删除书号为 " + no + " 的图书吗？")) {
```

```
            return;
        }
        try {
            service.removeBook(no);
            UIUtil.info(this, "图书删除成功。");
            clearForm();
            reload();
        } catch (Exception ex) {
            UIUtil.error(this, "删除失败: \n" + ex.getMessage());
        }
    }
public void delete(String bookNo) throws SQLException {
        try (Connection conn = DBUtil.getConnection();
             CallableStatement cs = conn.prepareCall("{call sp_book_delete(?)}")) {
            cs.setString(1, bookNo);
            cs.execute();
        }
    }
```

删除模块体现了“把规则放在数据库层”的思路。界面上看不到“有没有未归还”“有没有历史记录”的判断代码，程序只是调用存储过程，数据库拒绝时把消息显示出来。这样即使以后增加网页端或手机端，删除规则依然有效，不会因为换了一个入口就出现违反规则的数据。读者删除模块同样由 sp_reader_delete 和外键 fk_borrow_reader 保护，消息为“该读者尚有图书未归还，不能删除”。

4.5 数据库端存储过程的实现

四个模块的写操作最终都落到存储过程上，存储过程本身也是需要按规范编写的代码。以图书信息的修改和删除为例，存储过程用 IN 参数接收数据，用 WHERE 主键定位记录；删除过程在 DELETE 之前先做业务检查，检查不通过时用 SIGNAL SQLSTATE '45000' 抛出错误，MySQL 会把 MESSAGE_TEXT 原样传给客户端。

```
CREATE PROCEDURE sp_book_update(IN p_no VARCHAR(20), IN p_title VARCHAR(120),
                                IN p_author VARCHAR(60), IN p_price DECIMAL(8, 2),
                                IN p_pub VARCHAR(80), IN p_summary VARCHAR(500))
BEGIN
    UPDATE book
    SET title = p_title, author = p_author, price = p_price,
        publisher = p_pub, summary = p_summary
    WHERE book_no = p_no;
END$$
```

```
CREATE PROCEDURE sp_book_delete(IN p_no VARCHAR(20))
BEGIN
    IF EXISTS (SELECT 1 FROM borrow WHERE book_no = p_no AND return_date IS NULL) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '该图书仍在借阅中，不能删除';
    END IF;
    DELETE FROM book WHERE book_no = p_no;
END$$
```

存储过程代码同样遵守命名规范：过程名以 sp_ 开头、动词在前，参数以 p_ 开头（p_no、p_title），局部变量以 v_ 开头（v_out）。这样在几百行的建库脚本中，一眼就能区分表名、列名、过程名和参数名。

4.6 三角形判定程序代码

三角形判定程序 TriangleClassifier 共 142 行，包含常量定义、范围校验方法 validate、判定方法 classify、命令行入口 main 和输入读取方法 readThreeIntegers。完整代码如下。

```
package com..triangle;


import java.util.Scanner;


/**
 * 实验三：三角形判定程序。
 *
 * <p>判定逻辑严格遵循“先输入校验与取值范围、再三角形不等式、最后等边→等腰→一般”的顺序：
 * <ol>
 *   <li>输入个数必须是 3 个，且每个都必须能解析为整数；</li>
 *   <li>三条边都必须落在 [1, 100] 闭区间内；</li>
 *   <li>任意两边之和必须严格大于第三边，否则不构成三角形；</li>
 *   <li>三边全等为等边三角形；恰有两边相等为等腰三角形；三边互不相等为一般三角形。</li>
 * </ol>
 *
 * <p>除了 {@link #main(String[])} 之外，本类还提供 {@link #validate(int, int, int)} 与
 * {@link #classify(int, int, int)} 两个 public static 方法，便于 TriangleTest 直接调用做表驱动测试。
 */
public final class TriangleClassifier {


    /** 边长的下界（含）。 */
    public static final int MIN_SIDE = 1;
    /** 边长的上界（含）。 */
    public static final int MAX_SIDE = 100;
```

```
    /** 等边三角形。 */
    public static final String EQUILATERAL = "等边三角形";
    /** 等腰三角形。 */
    public static final String ISOSCELES = "等腰三角形";
    /** 一般三角形。 */
    public static final String SCALENE = "一般三角形";
    /** 不构成三角形。 */
    public static final String NOT_TRIANGLE = "不构成三角形";
    /** 输入越界（边长不在 [1, 100] 内）。 */
    public static final String OUT_OF_RANGE = "输入越界";

    private TriangleClassifier() {
        // 工具类，禁止实例化
    }

    /**
     * 校验三条边的取值范围。
     *
     * @return 三边均落在 [1, 100] 时返回 {@code null}；否则返回第一条越界边的说明文字
     */
    public static String validate(int a, int b, int c) {
        if (a < MIN_SIDE || a > MAX_SIDE) {
            return "边 a=" + a + " 超出 " + MIN_SIDE + "~" + MAX_SIDE + " 范围";
        }
        if (b < MIN_SIDE || b > MAX_SIDE) {
            return "边 b=" + b + " 超出 " + MIN_SIDE + "~" + MAX_SIDE + " 范围";
        }
        if (c < MIN_SIDE || c > MAX_SIDE) {
            return "边 c=" + c + " 超出 " + MIN_SIDE + "~" + MAX_SIDE + " 范围";
        }
        return null;
    }

    /**
     * 判定三条边构成的三角形类型。
     *
     * <p>判定顺序（不可交换）：范围校验 → 三角形不等式 → 等边 → 等腰 → 一般。
     *
     * @return 等边三角形 / 等腰三角形 / 一般三角形 / 不构成三角形 / 输入越界 之一
     */
    public static String classify(int a, int b, int c) {
        // 第一步：范围校验。必须在三角形不等式之前，否则负数、0 等非法输入可能被误判。
        if (validate(a, b, c) != null) {
```

```
            return OUT_OF_RANGE;
        }
        // 第二步：三角形不等式。用“和 <= 第三边”判否，可覆盖退化三角形（如 1,2,3）。
        // 采用 long 运算防止极端输入下 int 加法溢出。
        if ((long) a + b <= c || (long) a + c <= b || (long) b + c <= a) {
            return NOT_TRIANGLE;
        }
        // 第三步：等边 → 等腰 → 一般。
        if (a == b && b == c) {
            return EQUILATERAL;
        }
        if (a == b || b == c || a == c) {
            return ISOSCELES;
        }
        return SCALENE;
    }

    /** 命令行入口：从标准输入读取 3 个整数并输出判定结果。 */
    public static void main(String[] args) {
        int[] sides = readThreeIntegers();
        int a = sides[0];
        int b = sides[1];
        int c = sides[2];

        System.out.println("输入：a=" + a + ", b=" + b + ", c=" + c);

        String rangeError = validate(a, b, c);
        if (rangeError != null) {
            System.out.println("输入错误：" + rangeError + "（要求 " + MIN_SIDE + "~" + MAX_SIDE + "）");
            System.exit(1);
        }

        String result = classify(a, b, c);
        System.out.println("判定结果：" + result);
        if (NOT_TRIANGLE.equals(result)) {
            System.out.println("说明：三边不满足任意两边之和大于第三边。");
        }
        System.exit(0);
    }

    /**
     * 用 Scanner 从标准输入读取恰好 3 个整数。
     *
```

```
     * <p>出错时打印原因并以非 0 退出码结束：2 = 输入个数不对，3 = 存在非整数。读到合法三边时返回数组。
     */
    private static int[] readThreeIntegers() {
        Scanner scanner = new Scanner(System.in);
        int[] sides = new int[3];
        int count = 0;
        while (count < 3 && scanner.hasNext()) {
            String token = scanner.next();
            try {
                sides[count] = Integer.parseInt(token);
            } catch (NumberFormatException ex) {
                System.out.println("输入错误：\"" + token + "\" 不是整数");
                System.out.println("用法：依次输入 3 个整数 a、b、c（每边 " + MIN_SIDE + "~" + MAX_SIDE + "）");
                System.exit(3);
            }
            count++;
        }
        if (count < 3) {
            System.out.println("输入错误：需要输入 3 个整数（a、b、c），实际只读到 " + count + " 个");
            System.out.println("用法：依次输入 3 个整数 a、b、c（每边 " + MIN_SIDE + "~" + MAX_SIDE + "）");
            System.exit(2);
        }
        if (scanner.hasNext()) {
            String extra = scanner.next();
            System.out.println("输入错误：只需要 3 个整数，多余的输入为 \"" + extra + "\"");
            System.out.println("用法：依次输入 3 个整数 a、b、c（每边 " + MIN_SIDE + "~" + MAX_SIDE + "）");
            System.exit(2);
        }
        return sides;
    }
}
```

程序把“判定”和“输入输出”分开：classify 只接收三个 int 参数并返回结果字符串，不读输入、不打印，因此可以被 TriangleTest 直接调用做表驱动测试；main 负责读入、调用、输出和退出码。这种拆分让核心逻辑可以脱离控制台单独测试，是编码设计中的一条重要经验。

4.7 编译、运行与测试方法

图书管理系统工程的编译与运行方法见实验二报告，这里只给出三角形程序的命令。全部源码只依赖 JDK 标准库，用 javac 编译、java 运行即可；测试程序 TriangleTest 与主程序放在同一个包中，编译后可直接运行。

```
:: 在 experiment3 目录下编译
javac -encoding UTF-8 -d out src\com\\triangle\*.java


:: 运行主程序（从键盘输入三个整数）
java -cp out com..triangle.TriangleClassifier


:: 运行表驱动测试
java -cp out com..triangle.TriangleTest
```

主程序还支持从标准输入管道读取数据，便于批量验证和自动化测试。例如在 PowerShell 中执行 ″3 4 5″ | java -cp out com..triangle.TriangleClassifier，程序会输出“一般三角形”并以退出码 0 结束；输入越界时退出码为 1，输入个数不对时为 2，存在非整数时为 3。用退出码区分失败原因，是命令行程序约定俗成的做法，测试脚本可以据此判断执行结果。

五、结果分析

5.1 图书管理系统模块的功能测试

模块代码不能只看是否编译通过，还要验证它和数据库、存储过程、触发器配合起来是否按预期工作。为此编写了命令行测试程序 SmokeTest，它不经过图形界面，直接调用业务层接口，依次检查数据库连接、三张表的数据量、模糊查询、视图查询、存储过程的录入—修改—删除、触发器对重复借出的拦截以及主键冲突的报错。运行结果如下。

```
总览[图书,读者,借阅,未还] = [24, 18, 128, 24]
图书全部      = 24
图书查询'数据' = 2
读者全部      = 18
借阅明细      = 128
未归还        = 24
借阅排行前3   = 3
    数据结构(C语言版) / 清华大学出版社 / 借阅 8 次 / 在借 1
    离散数学 / 高等教育出版社 / 借阅 7 次 / 在借 1
    大学英语精读(第三册) / 上海外语教育出版社 / 借阅 7 次 / 在借 1
读者统计:
    学生 本科 人数=7 借阅=53 人均=7.57
    学生 硕士 人数=4 借阅=40 人均=10.0
    教师 教职工 人数=4 借阅=25 人均=6.25
    学生 博士 人数=2 借阅=7 人均=3.5
    职工 教职工 人数=1 借阅=3 人均=3.0
出版社种类    = 14 首位=高等教育出版社=32
月度统计      = {2026-03=30, 2026-04=21, 2026-05=19, 2026-06=18, 2026-07=13, 2026-08=19, 2026-09=8}
录入后查询    = 1
```

```
修改后        = Java 核心技术 卷 I / 159.00
触发器拦截    = 该图书尚未归还，不能重复借出
主键拦截      = Duplicate entry 'B0001' for key 'book.PRIMARY'
删除后查询    = 0
总览[图书,读者,借阅,未还] = [24, 18, 128, 24]
SMOKE_OK
```

测试结果表明四个模块的代码工作正常：查询按关键字“数据”筛出 2 条记录；录入书号 B0099 后能查到 1 条，修改后书名和价格同步更新，删除后又回到 0 条，说明 sp_book_insert、sp_book_update、sp_book_delete 三个存储过程都被正确调用；把尚未归还的图书再次借出时，触发器返回“该图书尚未归还，不能重复借出”；录入已存在的书号 B0001 时，主键约束返回 Duplicate entry 错误；测试结束后总览仍为 [24, 18, 128, 24]，说明测试没有污染数据库。最后输出 SMOKE_OK，表示全部检查通过。

界面层的四个模块与命令行测试共用同一套业务方法和数据访问方法，因此命令行测试通过，基本可以说明界面上的录入、删除、修改、查询也能正常工作。图 2 是图书管理模块的运行界面：上方是查询工具条，中间是图书列表，下方是录入 / 修改 / 删除表单；单击表格中的某一行，该行数据会自动回填到表单，修改后单击“修 改”即可保存。

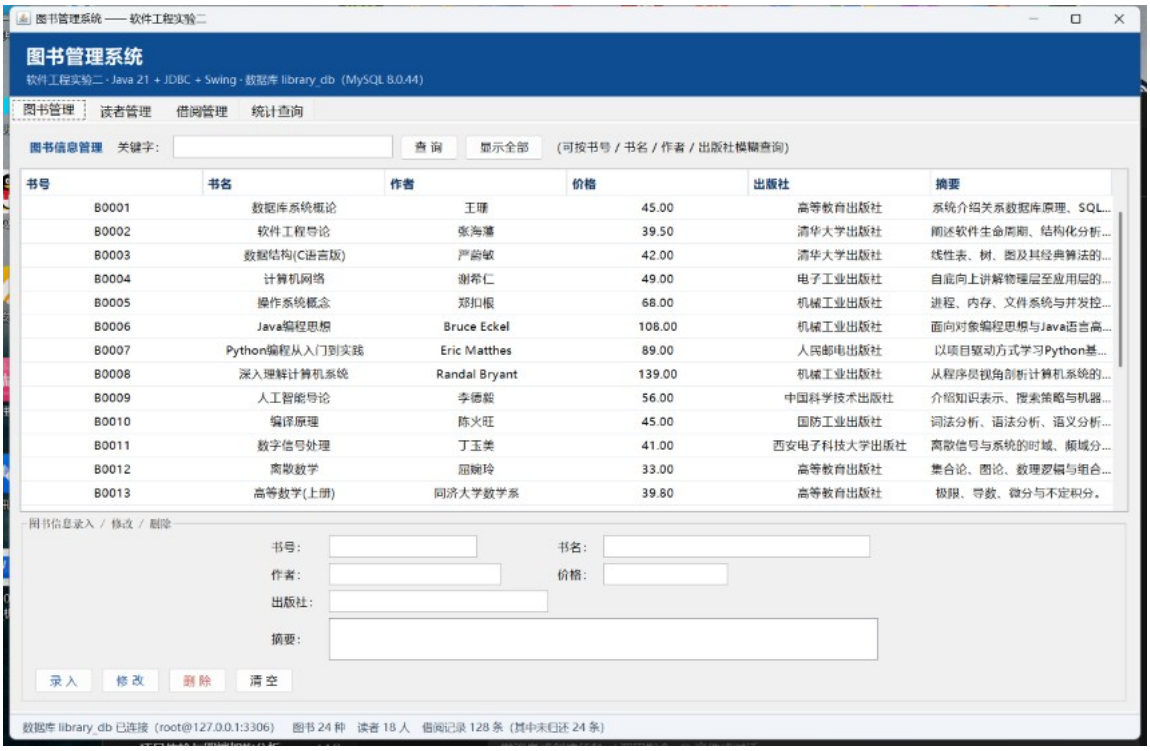


图 2　图书管理模块运行界面

5.2　三角形判定程序的测试

三角形程序的测试采用表驱动方式：把每条用例的输入、期望结果和覆盖点写进数组，由 TriangleTest 逐条调用 classify 并比较实际结果，全部通过时退出码为 0。测试用例按等价

类和边界值设计，共 22 条，覆盖三种有效三角形、越界、非整数、输入个数错误和非三角形等情况，具体如表 6 所示。

表 6　三角形判定程序测试用例与执行结果

编号	输入 (a,b,c)	覆盖点	期望结果	结果
TC01	(5, 5, 5)	等价类：合法输入—三边全等（常规值）	等边三角形	PASS
TC02	(1, 1, 1)	边界值：三边均取下界 1	等边三角形	PASS
TC03	(100, 100, 100)	边界值：三边均取上界 100	等边三角形	PASS
TC04	(5, 5, 8)	等价类：合法输入—a=b 的等腰三角形	等腰三角形	PASS
TC05	(5, 8, 5)	等价类：a=c 的等腰三角形（输入顺序无关）	等腰三角形	PASS
TC06	(8, 5, 5)	等价类：b=c 的等腰三角形（输入顺序无关）	等腰三角形	PASS
TC07	(1, 2, 2)	边界值：最短边为 1，1+2>2 恰好成立	等腰三角形	PASS
TC08	(1, 100, 100)	边界值：最短边 1 与两条上界边，1+100>100 成立	等腰三角形	PASS
TC09	(3, 4, 5)	等价类：合法输入—三边互不相等（典型勾股数）	一般三角形	PASS
TC10	(5, 3, 4)	等价类：三边互不相等且输入顺序不同	一般三角形	PASS
TC11	(2, 3, 4)	等价类：三边互不相等的较小取值	一般三角形	PASS
TC12	(100, 99, 98)	边界值：接近上界的合法一般三角形	一般三角形	PASS
TC13	(1, 2, 3)	边界值：a+b=c，退化三角形（经典非三角用例）	不构成三角形	PASS
TC14	(1, 1, 2)	边界值：等腰退化 a+b=c，不构成三角形	不构成三角形	PASS
TC15	(1, 2, 4)	等价类：a+b<c，两边之和小于第三边	不构成三角形	PASS
TC16	(100, 50, 50)	边界值：50+50=100，大数下的退化三角形	不构成三角形	PASS

TC17	(100, 1, 1)	边界值：两条最短边之和远小于最长边	不构成三角形	PASS
TC18	(0, 3, 4)	边界值：下边界外一点 0	输入越界	PASS
TC19	(101, 50, 50)	边界值：上边界外一点 101	输入越界	PASS
TC20	(-3, 4, 5)	等价类：负整数输入	输入越界	PASS
TC21	(0, 0, 0)	等价类：全零输入	输入越界	PASS
TC22	(100, 100, 1)	边界值：两条上界边与最短边 1，构成等腰三角形	等腰三角形	PASS

22 条用例全部通过（PASS 22 / FAIL 0）。除表 6 的用例之外，测试程序还做了两项附加校验：一是把 (3,4,5)、(5,5,8)、(1,2,2) 等用例的三条边做全排列，共 108 次判定，结果与期望全部一致，说明程序不依赖输入顺序；二是对 22 条用例比较 validate 与 classify 的越界判断是否一致，22 条全部一致。最后输出如下。

```
用例汇总：共 22 条，PASS 22 条，FAIL 0 条
附加校验 1（输入顺序无关性/全排列）：108/108 次判定与期望一致
附加校验 2（validate 与 classify 越界判定一致性）：22/22 条一致
结论：全部测试通过（退出码 0）
```

主程序的运行记录也一并保存了下来。下面摘录 4 个有代表性的场景：正常的一般三角形、两边之和等于第三边的退化三角形、越界输入和非整数输入。

```
$ "3 4 5" | java -cp experiment3\out com..triangle.TriangleClassifier
输入：a=3, b=4, c=5
判定结果：一般三角形
[退出码] 0

$ "1 2 3" | java -cp experiment3\out com..triangle.TriangleClassifier
输入：a=1, b=2, c=3
判定结果：不构成三角形
说明：三边不满足任意两边之和大于第三边。
[退出码] 0

$ "0 3 4" | java -cp experiment3\out com..triangle.TriangleClassifier
输入：a=0, b=3, c=4
输入错误：边 a=0 超出 1~100 范围（要求 1~100）
[退出码] 1

$ "abc" | java -cp experiment3\out com..triangle.TriangleClassifier
输入错误："abc" 不是整数
```

```
用法：依次输入 3 个整数 a、b、c（每边 1~100）
[退出码] 3
```

从结果看，程序对四类情况给出了不同的反馈：合法输入输出三角形类型并以 0 退出；退化三角形判为“不构成三角形”；越界输入报告是哪一条边超出范围并以 1 退出；非整数输入报告原始 token 并以 3 退出。特别值得一提的是 1、2、3 和 1、1、2 这两组用例：它们很容易因为判定条件写成“小于”而不是“小于等于”而被误判为一般三角形或等腰三角形，测试用例把这一缺陷直接暴露了出来。

5.3 代码质量分析

为了对代码质量做定量评价，编写脚本统计了 library-app 应用源代码的规模、注释、方法长度、分支数和命名规范，结果如表 7 所示。

表 7 图书管理系统源代码度量结果

度量项	数值	说明
应用源文件 / 类	19 / 20	src/main/java 下 19 个 .java，含 1 个内部类
方法数	174	不含构造方法
物理行（含空行）	2179	代码 1789 行 + 注释 87 行 + 空行 303 行
注释率	4.64%	整行注释 ÷（代码行 + 整行注释）
Javadoc 覆盖	20/20 个类，26/174 个方法	类全部有 Javadoc，方法级覆盖偏低
方法长度 / 复杂度	7.84 行 / 1.58	平均方法长度 / 平均圈复杂度；最长 75 行、最大 8（BarChartPanel.paintComponent）
规范与坏味道	违规 0 / 坏味道 7	命名检查全部通过；printStackTrace 0、System.out 0、未用 import 2、空 catch 3（DBUtil 有意忽略）、未调用方法 2

表 7 说明，程序在结构性指标上表现较好：174 个方法平均只有 7.84 行，平均圈复杂度 1.58，最大的方法是最长 75 行的 paintComponent（Java2D 绘图方法，分支较多但逻辑线性），没有命名违规，也没有调试输出残留；文件、类、方法三个层次的划分与五个包的对应关系清楚。图 3 进一步给出了各包的代码量构成，ui 包 7 个文件共 1095 行，占全部代码的一半以上，符合 Swing 界面代码量大的特点；dao 包 4 个文件共 428 行，集中了全部 SQL 和存储过程调用；entity 包以 getter/setter 为主，注释很少是正常现象。

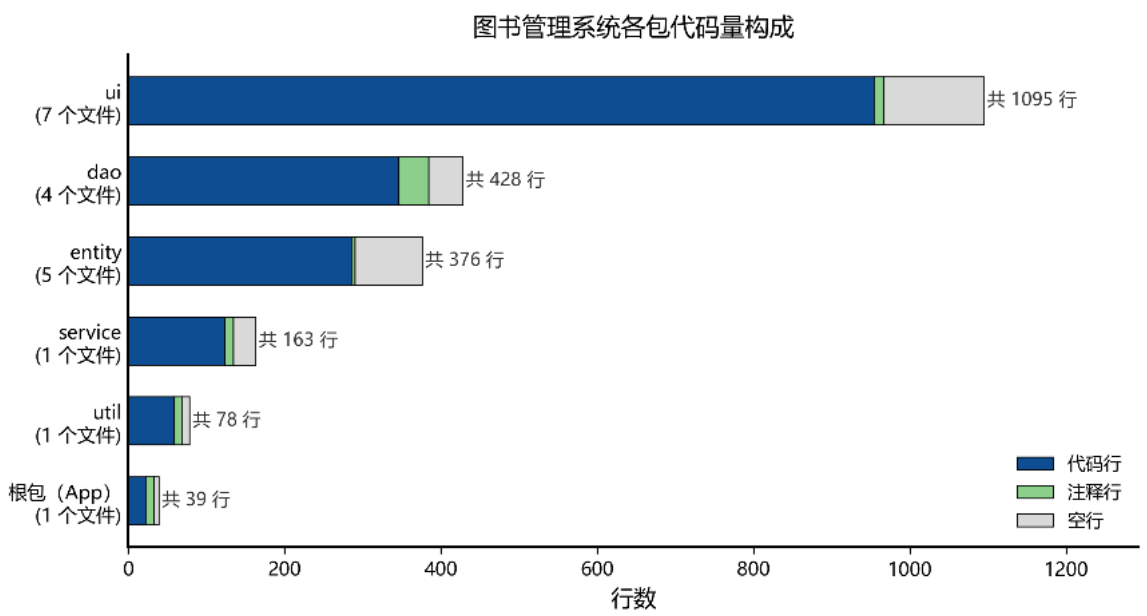


图 3　图书管理系统各包代码量构成

度量也暴露出几个不足。第一，注释率只有 4.64%，方法级 Javadoc 覆盖 26/174，尤其是 ui 包的几个面板类注释率不足 1%，这与 3.2 节提出的“类与公有方法写 Javadoc”的要求有差距；界面代码虽然直观，但事件方法较多，缺少注释会让后来者难以判断每个按钮的职责。第二，BookPanel、StatPanel 各残留 1 处未使用的 import，DBUtil 中两个 close 重载已无调用点，属于迭代过程中留下的冗余代码，应删除。第三，DBUtil 有 3 处空 catch，虽然是有意忽略关闭异常，但没有写注释说明理由，容易被误认为漏处理异常，应补充说明。

作为对照，三角形程序的 TriangleClassifier 共 142 行、TriangleTest 共 218 行。主程序把输入读取、范围校验、类型判定拆成三个方法，平均方法长度不到 20 行；测试程序用数组组织用例，新增用例只要加一行数据。两个程序的共同点是：核心逻辑短小、分支清晰、异常路径可测。这也说明代码质量与程序规模没有必然联系，小程序的注释和测试同样需要认真对待。

5.4　实验结论与心得体会

1. 编码规范不是形式主义，它直接决定代码能否被维护。本次实验中，命名规范检查 0 违规、包结构与分层一一对应，使得四个模块的调用链可以画成一张表；相反，注释率只有 4.64%、方法级 Javadoc 覆盖不足，说明规范执行得并不均衡，界面层尤其欠缺。规范只有落实到每一层、每一类文件，才真正有效。

2. 编程风格与选题质量相辅相成。图书管理系统规模适中，足以练到分层、异常处理和数据库访问；三角形程序规模很小，却把输入校验、边界值和分支顺序三个最容易出错的点集中暴露出来。选题过大容易只顾功能，选题过小容易忽略结构，本次两项任务正好互补。

3. 测试用例的设计比测试本身更重要。1、2、3 与 1、1、2 两组退化三角形用例，专门检

验“小于等于”这个容易写错的边界；全排列校验则验证了判定与输入顺序无关。22 条用例全部通过、108 次全排列一致，说明程序逻辑是可靠的；如果只测 3、4、5，这个缺陷很可能被漏掉。

4. 把完整性约束放在数据库层是正确的分工。界面层拦截空值和格式错误，业务层拦截业务规则，数据库层用主键、外键、ENUM、触发器和存储过程兜底。程序没有在 Java 中重复实现数据库的检查，既避免了规则不一致，也让四个模块的代码保持简短。

5. 程序仍有可改进之处：一是补充界面层注释和方法级 Javadoc，清理未使用的 import 和 DBUtil 的冗余重载；二是 update 和 delete 没有检查影响行数，书号输入错误时 UPDATE 会静默失败，可以在 DAO 中判断返回值或先查询记录是否存在；三是 LIKE 查询没有转义 % 和 _ 通配符；四是每次操作都新建数据库连接，数据量大时可引入连接池；五是测试目前是自编的表驱动程序，后续可以改用 JUnit 并接入 JaCoCo 统计语句覆盖率。

6. 通过本次实验，我对“编码设计”有了更完整的认识：编码不是把设计翻译成能运行的语句，而是在命名、结构、注释、异常、测试各方面都做出取舍的过程。先定规范再写代码，写完代码立刻测试边界，测试通过后再回头审查注释和冗余，这三步缺一不可。

教师签字		年　　月 日	